PhantomChannel: Artifact Appendix

Xu Zhang, Ming Zhou, Meng Zhang, Dong Wei, Haining Wang, and Qiaoyu Zhang

A ARTIFACT APPENDIX

A.1 Abstract

PhantomChannel is a research prototype for embedding auxiliary symbols beyond BLE packet parsing boundaries and recovering them with an SDR receiver. The artifact supplies transmitter and receiver source, firmware images, tests, demonstration videos, and an offline experiment bundle. This appendix describes installation, execution, output inspection, and the scope of the supported replays. Project-owned source is MIT licensed; videos are CC BY 4.0. Third-party source and firmware retain their file-level and vendor terms; consult `THIRD_PARTY_NOTICES.md` before rebuilding or redistribution.

A.2 Description & Requirements

The source archive provides `receiver/`, `firmware/`, `patches/zephyr/`, `configs/`, `tools/`, and `tests/`. The experiment archive provides `data/`, `scripts/`, `checks/`, and `reproduce_all.py`. `DEPENDENCY_MAP.md` maps analysis inputs to outputs. Each archive has a README. Generated CSV/JSON files are written under `outputs/`; plots are written to the experiment root and `plots/`. The workflow reanalyzes retained records rather than performing new RF acquisition.

A.2.1 Security, privacy, and ethical concerns. Offline execution requires no privileged access or radio transmission. Hardware evaluation replaces board firmware and emits BLE traffic. Use owned devices and authorized, isolated RF setups under applicable spectrum rules. Keep credentials and device identifiers out of shared configurations and captures. Published packet records use anonymous groups and relative timing.

A.2.2 How to access. The archived artifact is available at doi:10.5281/zenodo.22842761. Download the source and experiment ZIPs with suffix `2026.09.19-ae`, the top-level README, and `SHA256SUMS`. Verify downloaded files using `sha256sum --ignore-missing -c SHA256SUMS`. The maintained repository is https://github.com/Tincyu/PhantomChannel. Its `reproduction/` directory supplies the supplemental `requirements-verified.txt` and `ENVIRONMENT.md`; its `appendix/` directory supplies `verify_antenna_correctness.py`. Record the repository commit used for these supplemental files. The archived experiment ZIP fixes the inputs and scripts for the replay described here.

A.2.3 Hardware dependencies. Offline replay uses a conventional CPU workstation with sufficient memory and working disk space for extraction and generated plots. It requires no SDR, GPU, or BLE board. Optional live evaluation uses a supported BLE transmitter and peer together with a USRP B210 or X310 receiver. Supported transmitter families include Nordic, DA14695, EFR32MG24, CC2340R5, and STM32 with SX1280. Select boards and images using `firmware/images/FIRMWARE_GUIDE.md`; follow the platform-specific programming and runtime instructions there.

A.2.4 Software dependencies. The verified analysis environment is Ubuntu 24.04.4 LTS with Python 3.12.3, NumPy 2.5.4, SciPy 1.18.1, and Matplotlib 3.11.2. The supplemental `requirements-verified.txt` pins the full installed analysis package set; `ENVIRONMENT.md` documents installation and environment capture. Source tests use the archive's separate `requirements-test.txt`. Live reception additionally requires UHD and the selected receiver runtime. Firmware builds require external vendor SDKs and toolchains; the firmware guide and controller patch notes specify the corresponding versions and base revisions. Vendor SDKs are not bundled.

A.2.5 Benchmarks. The experiment archive includes packet IQ windows, parser candidates, advertising records, connected timing vectors, bandwidth summaries, Data ablation records, detector features and scores, and auxiliary NLOS and antenna records. These are experiment-specific workloads. Table 4 replay covers the Data portion. Real-boundary PIP summaries are archived diagnostics rather than independently regenerated packet-score evidence in this workflow.

A.3 Set Up

A.3.1 Installation. Extract the source and experiment ZIPs into separate directories. Copy the supplemental `requirements-verified.txt` into the extracted experiment root. In the source root, run:

```
python3 -m venv .venv
.venv/bin/python -m pip install \
  -r requirements-test.txt
```

In the experiment root, run:

```
python3 -m venv .venv
.venv/bin/python -m pip install \
  -r requirements-verified.txt
.venv/bin/python -m pip check
```

Installation needs network access or a local wheel cache. Subsequent replay is offline. Keep the environments separate and run each command from the corresponding archive root. Save `python --version`, OS information, and `pip freeze` with the run, as detailed in the environment guide.

A.3.2 Basic test. In the source root, run `.venv/bin/python -m pytest -q tests`. Success means the test command exits successfully with no failed tests. In the experiment root, run `.venv/bin/python scripts/recompute_tables.py`. Expect successful completion and table CSVs under `outputs/recomputed_tables/`. These checks establish software readiness; experiment interpretation follows the scope and definitions below.

A.4 Evaluation Workflow

A.4.1 Major claims.

(C1) The receiver and auxiliary analyses support inspection of the recorded bandwidth, range, SNR, and BPL/CAF evaluation (paper Fig. 10 and Tables 3–4; E1), using explicitly defined frame-recovery metrics.

(C2) The timing analyses characterize protocol-visible timing for the retained packet groups and selected samples (paper Fig. 12 and Table 5; E2).

(C3) The spectral and tail-energy analyses characterize observability using retained IQ windows, eligible packet features, and supported session-held-out score views (paper Fig. 11 and countermeasure evaluation; E3).

A.4.2 Experiments. In the experiment root, run:

```
.venv/bin/python reproduce_all.py
```

Expect successful completion, `[OK]` status messages, and generated plots and result files. Regression checks establish agreement with the archived analysis baseline; they are not an independent confirmation of every value printed in the paper. As a planning estimate, allow about half an hour for setup, a few person-minutes to inspect each experiment group, and several compute-minutes per group on a conventional workstation; dependency downloads and host performance affect elapsed time.

(E1) **Receiver and auxiliary replay (C1).** *Preparation:* retain the supplied range, bandwidth, ablation, and auxiliary records, together with the archived thresholds and run-selection rules. *Execution:* run the complete workflow above. *Results:* inspect the overhead, range, and SNR plots, the bandwidth and Data-ablation CSVs under `outputs/recomputed_tables/`, and the outputs under `outputs/auxiliary_experiments/`. The dependency map identifies each input and analysis entry point.
Interpret candidate recovery, full-frame rate, and correct useful-payload rate separately. Candidate recovery counts unique sequence representatives against the recorded nominal opportunities. Full-frame rate includes framing bytes and need not require integrity acceptance. Correct useful-payload rate requires known-pattern agreement and integrity acceptance, excludes framing and the fixed marker, and uses capture duration as the time denominator. After replay, run the companion `verify_antenna_correctness.py` with the extracted experiment root as its argument to inspect the stricter antenna metrics. Nominal opportunities are not independent transmitter ground truth; this correctness comparison is confined to the antenna records.

(E2) **Timing replay (C2).** *Preparation:* preserve packet grouping, timing vectors, and the archived sample-selection rules. *Execution:* run the complete workflow, or `.venv/bin/python scripts/recompute_tables.py` for tables alone. *Results:* inspect the advertising timing plots and `outputs/recomputed_tables/table5_conn_timing.csv`. Use `Input_dataset` to trace each output condition to its input vector. Condition aliases describe archived datasets; they do not independently certify transmitted payload length. The summaries describe the retained, filtered samples and are not presented here as a reconstruction of the paper's historical timing means.

(E3) **Spectral and detector replay (C3).** *Preparation:* retain the IQ group identities, feature eligibility masks, and session grouping. *Execution:* run the complete workflow. Detector analysis alone uses `.venv/bin/python scripts/tail_energy.py`. *Results:* inspect `outputs/frequency_similarity.csv` and `outputs/frequency_pip_pairwise.csv`. The metric uses cosine similarity of Welch PSDs converted to dB and independently min–max normalized. PIP validation uses the cross-group pairwise statistic; an illustrative spectrum overlay is not evidence of that aggregate metric.
Inspect `outputs/tail_energy/advertising_formal_roc.csv` and `connected_session_oof.csv` in the same directory for feature-based advertising replay and the direct-tail/PIP outer-boundary session-held-out views. Treat `connected_pip_auc.csv` as an archived diagnostic summary. Real-boundary diagnostic configurations must be distinguished by guard, window, boundary mapping, and sample eligibility; they are not interchangeable with session-held-out validation. Independent real-boundary score reconstruction is outside the reproduction claim made here.

A.5 Notes on Reusability

Use `DEPENDENCY_MAP.md` and the archive READMEs to replace inputs while preserving schemas, grouping, sample masks, and metric definitions. Keep input checksums, environment manifests, and generated results together. Changing inputs or processing rules defines a new analysis and should be documented separately from the archived replay. Hardware/RF acceptance requires a separate live run using the firmware guide.

A.6 Version

Based on the LaTeX template for Artifact Evaluation V20220926.